

Utiliser les simulateurs de cycles de vie logiciel (SIMSE)

Atelier individuel de préparation

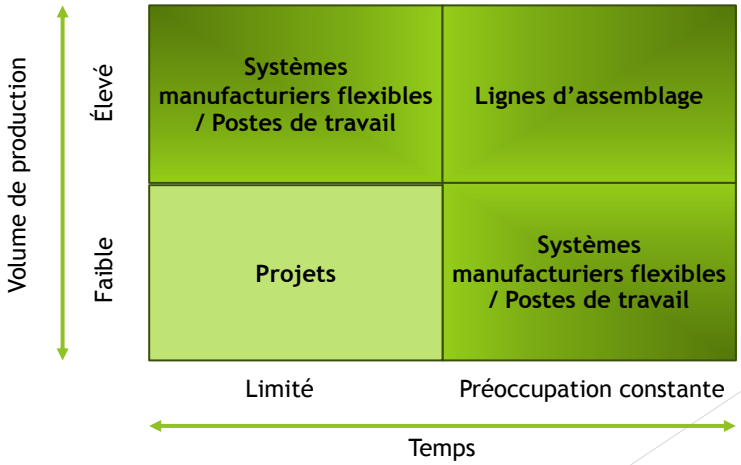
Par Sylvie Trudel, Ph.D., UQAM (tiré et traduit d'une présentation d'Alain April)

1

1- Aperçu de la théorie de gestion de projet logiciel

2

Un projet est une forme d'organisation du travail

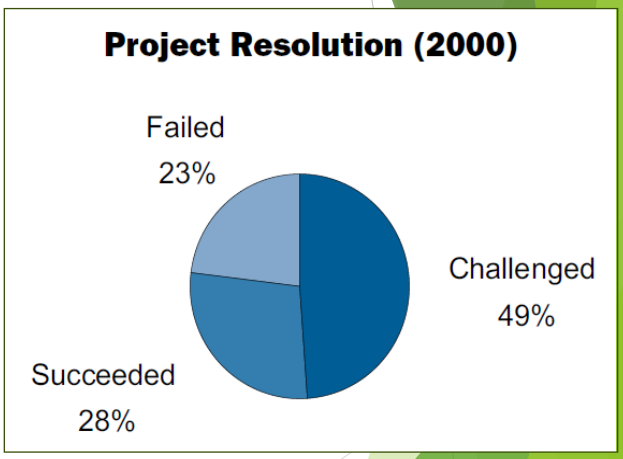


3

3

Plusieurs projets:

1)Ne livrent pas dans les délais;
2)Ne livrent pas dans le budget;
3)Ne livrent pas du tout!



4

4

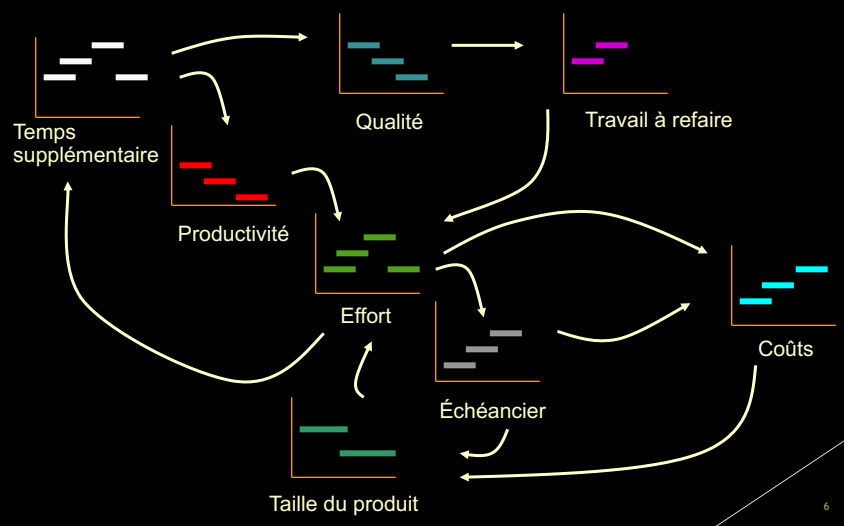
Comportements complexes de projets

- On ne peut pas les orienter dans la direction souhaitée en faisant juste une chose à la fois, que ce soit des heures supplémentaires, une réduction de la portée ou une augmentation des effectifs d'un projet, et il existe un risque omniprésent de « sur-orienter » le projet.
- Les réponses aux actions du chef de projet ne sont pas linéaires, décalées dans le temps et dépendent du temps. La démonstrabilité des causes et des effets est ambiguë. Les intrants et les extrants ne sont pas proportionnels. Le tout n'est pas quantitativement égal à ses parties, ni même qualitativement reconnaissable dans ses composants constitutifs.

5

5

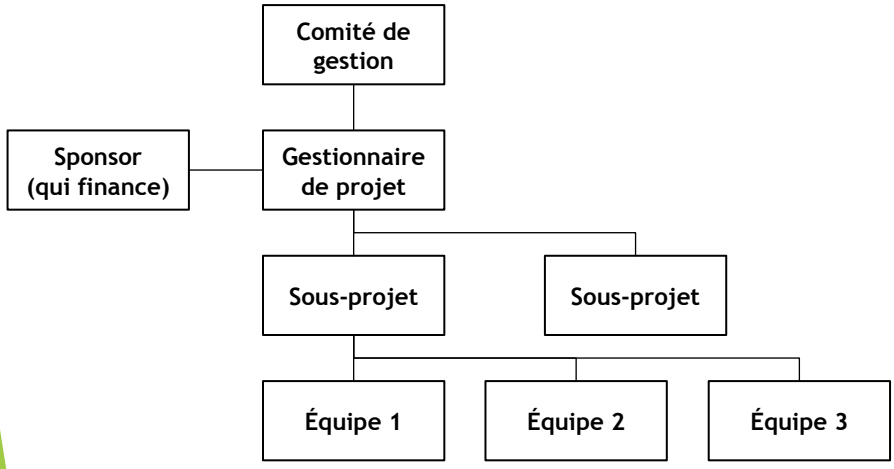
Dynamique de la gestion de projet



6

6

Rôles dans l'organisation d'un projet



7

7

Travaux de gestion de projet

- Qu'est-ce qui est fait?
  - Identifier les tâches à faire
  - Produire les estimations
  - Planifier travaux, ressources et financement
  - Identifier les problèmes et les risques
  - Acquérir les ressources
  - Établir un référentiel de performance
  - Mesurer l'avancement (progrès)
  - Prendre des actions correctives
  - Motiver
  - Communiquer
- Quand?
  - Au début du projet
  - Tel une vague déferlante
- Comment?
  - Système de répartition du travail (WBS)
  - Méthode avec un modèle d'estimation
  - Logiciel/technique de planification
  - Maintenir un carnet de risques et problèmes
  - Divers outils (ex. PERT, WBS, Gantt, etc.)
  - COSMIC ou autre méthode de taille fonctionnelle
  - Méthode EVM (Earned Value Management)
  - Comparer la performance et le plan puis ajuster
  - Choisir le cycle de développement (SDLC)
  - Etc…

8

8

Cycle de vie de développement logiciel (SDLC)

- Projet
  - une entreprise planifiée qui a un début et une fin, et qui produit un résultat ou un produit prédéterminé.
- Projet de développement de systèmes
  - Entreprise planifiée
  - Travail de grande envergure
  - Produit un nouveau système (nombreux artefacts + code source)
- Exigences du projet réussi
  - Plans détaillés
  - Séquence organisée et méthodique de tâches et d'activités

9

9

Ressources limitées et identification des tâches

- Un projet a des ressources et des contraintes limitées
  - Un **logiciel complet et sans erreur** doit être produit par une **équipe d'employés** dans un certain **calendrier** (clics dans les simulations) et un certain **budget**
- Produire un logiciel (interagir avec les employés)
  - Nécessite que le chef de projet **affecte intelligemment les employés** aux activités du SDLC
  - Tous les employés sont payés, ils doivent donc être affectés au travail à tout moment
  - L'utilisation des **outils de GL** aidera à produire des logiciels plus rapidement avec moins de défauts
  - Les employés ont une expérience spécifique, un taux de salaire, des niveaux d'énergie et d'humeur qui ont un impact sur leur productivité
  - Lorsque les employés travaillent, ils génèrent également des erreurs cachées qui devront être supprimées en affectant des employés à la **révision des artefacts** = trouver des erreurs pour les supprimer

10

10

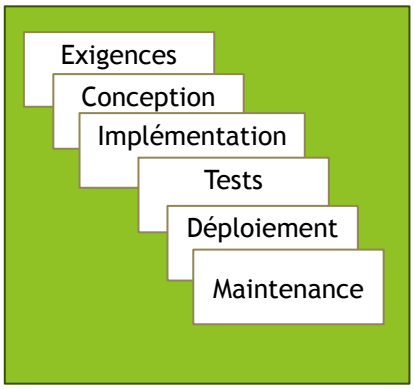
Choisir un SDLC va influencer les pratiques de gestion de projet

- SDLC : il s'agit d'un cadre qui décrit les activités réalisées à chaque étape d'un projet de développement logiciel ;
- Il existe de nombreux types parmi lesquels choisir :
  - Le modèle en cascade (Waterfall) ;
  - Le modèle incrémental ;
  - Le modèle de prototypage rapide (RAD);
  - SDLC agile :
    - Le modèle XP (Extreme Programming);
    - Le modèle RUP (Rational Unified Process).

11

11

Modèle en Cascade



- Documents d'exigences – définissent les informations requises, les fonctionnalités, les comportements, la performance et les interfaces.
- Documents de conception – structures des données, architecture logicielle, représentations des interfaces, détails algorithmiques.
- Code Source – base de données, documentation utilisateur (faite en parallèle).
- Tests – Plan des tests système, cas de tests, rapports d'erreurs, retester (réexécution des tests après corrections)

12

12

Forces et faiblesses du modèle Cascade

Forces

- **Facile à comprendre**, facile à utiliser
- **Fournit une structure** au personnel inexpérimenté
- **Les jalons sont bien compris**
- Établit la **stabilité des exigences**
- Idéal pour le **contrôle de gestion** (planification, dotation en personnel, suivi)
- Fonctionne bien lorsque la **qualité est plus importante** que le coût ou le calendrier

Faiblesses

- **Toutes les exigences doivent être connues** à l'avance
- Les livrables créés pour chaque phase sont considérés comme figés, **ce qui entrave la flexibilité**
- Peut donner une **fausse impression de progression**
- **Ne reflète pas la nature de résolution de problèmes** du développement logiciel, itérations de phases
- L'intégration est un **big bang à la fin**
- **Peu d'opportunités pour le client** de prévisualiser le système (jusqu'à ce qu'il soit trop tard)

13

13

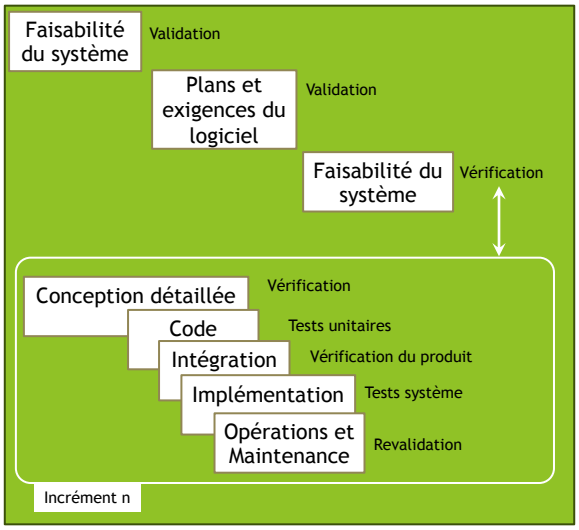
Quand utiliser le modèle Cascade?

- Les exigences sont **très bien connues**
- La définition du produit est **stable**
- La technologie est **comprise et maîtrisée**
- Nouvelle **version d'un produit existant**
- **Conversion d'un produit existant** vers une nouvelle plateforme.

14

14

Modèle Incrémental



- Construire une implémentation partielle d'un système total
- Puis ajouter lentement des fonctionnalités supplémentaires
- Le modèle incrémental hiérarchise les exigences du système, puis les implémente par groupes.
- Chaque version ultérieure du système ajoute des fonctions à la version précédente, jusqu'à ce que toutes les fonctionnalités conçues aient été implémentées.

15

15

Forces et faiblesses du modèle Incrémental

Forces

- Développer **d'abord les fonctions à haut risque ou majeures**
- Chaque version fournit **un produit opérationnel**
- Le client peut **répondre à chaque build**
- Utilise la **répartition des tâches** « diviser pour régner »
- Réduit le **coût initial de livraison**
- La livraison initiale du produit est **plus rapide**
- Les clients **obtiennent rapidement les fonctionnalités importantes**
- Le risque de **modification des exigences est réduit**

Faiblesses

- Nécessite une **bonne planification et une bonne conception**
- **Nécessite une définition précoce d'un système complet et entièrement fonctionnel** pour permettre la définition des incréments
- **Des interfaces de modules bien définies** sont nécessaires (certaines seront développées bien avant d'autres)
- Le coût total du système complet **n'est pas inférieur**

16

16

Quand utiliser le modèle Incrémental?

- Risque, financement, échéancier, complexité du programme ou nécessité de **concrétiser rapidement les avantages**.
- La plupart des exigences sont connues à l'avance, mais devraient **évoluer au fil du temps**
- Nécessité de **mettre rapidement sur le marché les fonctionnalités de base**
- Sur des projets dont **les échéanciers de développement sont longs**
- Sur un projet avec **une nouvelle technologie**.

17

17

Modèle *Rapid Application Development* (RAD)

- **Phase de planification des exigences** (un atelier utilisant une discussion structurée sur les problèmes commerciaux)
- **Phase de description de l'utilisateur** – des outils automatisés capturent les informations des utilisateurs
- **Phase de construction** – des outils de productivité, tels que des générateurs de code, des générateurs d'écran, etc. dans un délai imparti (« Faire jusqu'à ce que ce soit fait »)
- **Phase de basculement** – installation du système, tests d'acceptation des utilisateurs et formation des utilisateurs

18

18

Forces et faiblesses du modèle RAD

Forces

- **La réduction du temps de cycle** et l'amélioration de la productivité avec moins de personnel signifient des coûts inférieurs
- L'approche de la **boîte temporelle** atténue les risques liés aux coûts et au calendrier
- **L'implication du client tout au long** du cycle minimise le risque de ne pas atteindre la satisfaction du client et les besoins de l'entreprise
- L'accent est mis sur le code plutôt que sur la documentation (**WYSIWYG**).
- **Utilise des concepts de modélisation** pour capturer des informations sur l'entreprise, les données et les processus.

Faiblesses

- Le processus de développement accéléré **doit fournir des réponses rapides** à l'utilisateur
- Risque de **ne jamais parvenir à une clôture**
- Difficile à utiliser avec les **systèmes légataires**
- Nécessite un système qui peut être **modularisé**
- Les développeurs et les clients doivent **s'engager dans des activités rapides** dans un délai réduit.

19

19

Quand utiliser le modèle RAD?

- Exigences raisonnablement **bien connues**
- Utilisateur impliqué **tout au long du cycle de vie**
- Le projet peut **être limité dans le temps** (time-boxed)
- Fonctionnalités fournies par **incréments**
- **Haute performance non requise**
- **Faibles risques techniques**
- Le système **peut être modularisé**.

20

20

Le cycles de vie « Agile »

- Accélérer ou contourner une ou plusieurs phases du cycle de vie
- Généralement moins formel et à portée réduite (à discuter…)
- Utilisé pour les applications critiques en termes de temps
- Utilisé dans les organisations qui emploient des méthodes disciplinées

21

21

Quelques méthodes Agiles

- Adaptive Software Development (ASD)
- Feature Driven Development (FDD)
- Crystal Clear
- Dynamic Software Development Method (DSDM)

Ces méthodes sont peu utilisées en industrie

- Rapid Application Development (RAD) → Méthode plus rarement utilisée
- Scrum → La méthode la plus populaire
- Extreme Programming (XP) → Existait avant la publication du manifeste Agile
- Rational Unify Process (RUP) → Il existe une version Agile et une classique

22

22

Extreme Programming – (XP)

Pour les équipes de petite et moyenne taille qui développent des logiciels avec des exigences vagues ou en évolution rapide

La programmation (coding) est l'activité clé tout au long d'un projet logiciel

- La communication entre les coéquipiers se fait avec du code
- Cycle de vie et comportement d'objets complexes définis dans des cas de test - encore une fois dans le code

23

Les 12 pratiques de XP

1. **Jeu de planification** – déterminer la portée de la prochaine version en combinant les priorités commerciales et les estimations techniques
2. **Petites versions** – mettre un système simple en production, puis publier de nouvelles versions dans un cycle très court (incrémentales)
3. **Métaphore** – tout le développement est guidé par une histoire simple et partagée sur le fonctionnement de l'ensemble du système
4. **Conception simple** – le système est conçu aussi simplement que possible (la complexité supplémentaire est supprimée dès qu'elle est trouvée)
5. **Tests** – les programmeurs écrivent en permanence des tests unitaires ; les clients écrivent des tests pour les fonctionnalités
6. **Refactoring** – les programmeurs restructurent en permanence le système sans modifier son comportement pour supprimer les doublons et simplifier
7. **Programmation en binôme** : tout le code de production est écrit par deux programmeurs sur une seule machine
8. **Propriété collective** : n'importe qui peut modifier n'importe quel code n'importe où dans le système à tout moment.
9. **Intégration continue** : intégrer et construire le système plusieurs fois par jour, chaque fois qu'une tâche est terminée.
10. **Semaine de 40 heures** : ne pas travailler plus de 40 heures par semaine en règle générale
11. **Client sur site** : un utilisateur fait partie de l'équipe et est disponible à plein temps pour répondre aux questions
12. **Normes de programmation** : les programmeurs écrivent tout le code conformément aux règles mettant l'accent sur la communication via le code

24

XP est « extrême » parce que…

Pratiques de bon sens poussées à l'extrême:

- Si les révisions de code sont bonnes, **révisez le code tout le temps** (programmation en binôme)
- Si les tests sont bons, **tout le monde testera tout le temps**
- Si la simplicité est bonne, gardez le système dans la conception la plus simple qui supporte sa fonctionnalité actuelle. (**la chose la plus simple qui fonctionne**)
- Si la conception est bonne, tout le monde concevra quotidiennement (**refactoring**)
- Si l'architecture est importante, tout le monde travaillera à définir et à affiner l'architecture (**métaphore**)
- Si les tests d'intégration sont importants, construisez et **intégrez les tests plusieurs fois par jour** (intégration continue)
- Si les itérations courtes sont bonnes, **faites des itérations vraiment, vraiment courtes** (des heures plutôt que des semaines)

25

25

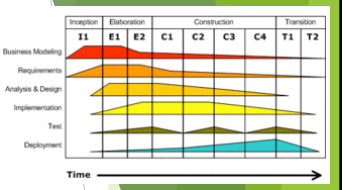
Modèle *Rational Unified Process* (RUP)

- Dans sa forme la plus simple, RUP se compose de quelques flux de travail fondamentaux :
  - **Ingénierie d'affaires** : comprendre les besoins de l'entreprise.
  - **Exigences** : traduire les besoins de l'entreprise en comportements d'un système automatisé.
  - **Analyse et conception** : traduire les exigences en architecture logicielle.
  - **Implémentation** : créer un logiciel qui s'intègre dans l'architecture et qui possède les comportements requis.
  - **Test** : s'assurer que les comportements requis sont corrects et que tous les comportements requis sont présents.
  - **Configuration et gestion des modifications** : suivre toutes les différentes versions de tous les produits de travail (incluant la documentation).
  - **Gestion de projet** : gérer les plannings et les ressources.
  - **Environnement** : configurer et maintenir l'environnement de développement.
  - **Déploiement** : tout ce qui est nécessaire pour déployer le projet.

26

26

Règles du modèle RUP: 6 meilleures pratiques



1. Développer itérativement
   - Il est préférable de connaître toutes les exigences à l'avance, mais ce n'est souvent pas le cas. Il existe plusieurs processus de développement de logiciels qui proposent des solutions pour minimiser les coûts en termes de phases de développement.
2. Gérer les exigences
   - Gardez toujours à l'esprit les exigences définies par les utilisateurs.
3. Utiliser des composantes
   - Décomposer un projet avancé n'est pas seulement suggéré, mais en fait inévitable.
4. Modéliser visuellement
   - Utilisez des diagrammes pour représenter tous les composants principaux, les utilisateurs et leurs interactions. « UML », abréviation de Unified Modeling Language, est un outil qui peut être utilisé pour rendre cette tâche plus réalisable.
5. Vérifier la qualité
   - Faites toujours des tests une partie importante du projet à tout moment. Les tests deviennent plus lourds à mesure que le projet progresse, mais doivent être un facteur constant dans toute création de produit logiciel.
6. Contrôler les changements
   - De nombreux projets sont créés par de nombreuses équipes, parfois dans des endroits différents, différentes plateformes peuvent être utilisées, etc. Par conséquent, il est essentiel de s'assurer que les modifications apportées à un système sont synchronisées et vérifiées en permanence. (Voir Intégration continue).

27

27

2- Pourquoi utiliser une simulation des cycles de vie de développement logiciel avec les étudiants?

28

28

Raisons pour que les étudiants appliquent les simulations de SDLC

- Les simulations permettent aux étudiants de découvrir comment chaque SDLC impacte leur style de gestion ;
- L'utilisation itérative du simulateur leur permet de tester différentes hypothèses et de voir le score obtenu ;
- C’est une excellente façon d'expérimenter/débattre des concepts abstraits intégrés dans la gestion de projets logiciels.

29

29

Introductions aux simulateurs SIMSE

- Un environnement de simulation d'ingénierie logicielle éducatif et ludique créé par Emily Navarro de l'Université de Californie – Irvine

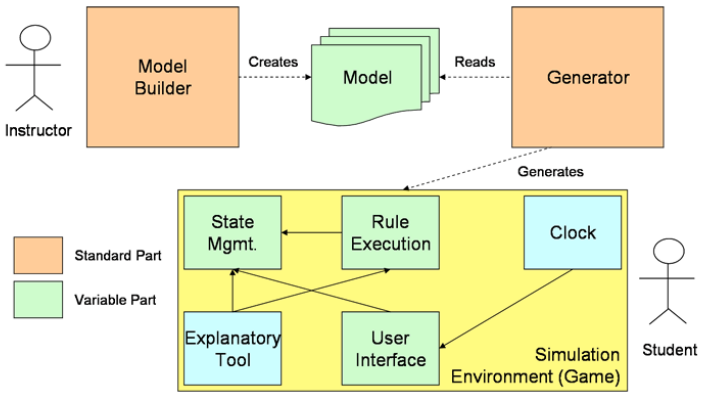
Cliquer ici pour accéder au site original

30

30

Introduction aux simulateurs SIMSE

- Comment sont-ils construits?



31

31

Introduction aux simulateurs SIMSE

- Quelle est la durée d'une simulation typique ?
  - 30 minutes à 2 heures pour le SDLC Waterfall
  - 20 minutes à une heure pour le SDLC RUP
  - 10 à 20 minutes pour le SDLC RAD
- Comment utiliser SIMSE dans ma classe ?
  - La manière la plus simple d'utiliser SimSE est d'assigner aux étudiants une simulation comme exercice à la maison. Bien sûr, étant l'approche la moins complexe, elle peut également offrir le moins d'avantages de toutes les approches.
  - Un instructeur peut également utiliser SimSE en classe, comme outil d'illustration pour les concepts enseignés dans un cours magistral. Par exemple, lorsqu'il parle de différents cycles de vie, un instructeur peut montrer des simulations SimSE de chaque modèle de cycle de vie. Bien sûr, cette option nécessiterait plus de temps et plus d'efforts pour réellement construire différents modèles, mais serait également plus gratifiante que des options plus simples.

32

32

Pour démarrer les simulateurs SIMSE…

- De quoi ai-je besoin pour exécuter le simulateur ?
  - Le dernier JDK (Java SE Development Kit)
- Comment obtenir le SDLC que je souhaite simuler ?
  - Chaque SDLC est prêt à l'emploi, il vous suffit de le télécharger depuis le site Web http://www.ics.uci.edu/~emilyo/SimSE/downloads.html
- Comment démarrer un jeu et comment fonctionne-t-il ?
  - Une fois un jeu téléchargé (.exe ou .jar), vous devez lire le fichier .txt qui expliquera l'objectif de votre jeu. Par exemple, l'objectif de Waterfall est :
  - Votre tâche consiste à créer un système WEB : Groceries@home
  - Votre budget : 280 000 $
  - Votre calendrier : 1 350 clics d'horloge
- Prêt ? Cliquez ensuite sur le fichier waterfall.exe pour démarrer le jeu

33

33

SIMSE: Avant de démarrer, écouter le tutoriel qui montre comment:

- Revoir les objectifs d'un jeu
- Afficher les ressources (**5 types** : artefacts, clients, employés, projets et outils)
- Affecter intelligemment un employé à une tâche
- Acheter un outil de génie logiciel (SE=Software Engineering) et l'utiliser
- Avancer dans le temps
- Être conscient des changements tout au long du jeu
- Consulter les informations sur les performances après avoir terminé un jeu

34

34

Suite, après la pause…

- Nous avons vu la théorie de la gestion de projet logiciel
- Nous avons été initiés à SIMSE

- Nous verrons les outils fournis pour mieux comprendre l'impact de notre décision de gestion de projet
- Et des exemples de résultats d'étudiants

35

35

SIMSE: Outil Explicatif

- Outils disponibles pour mieux comprendre l'effet de vos décisions pour :
  - Visualiser la représentation graphique (graphiques d'objets, graphiques d'actions et graphiques composites)
  - Découvrir des informations sur les règles sous-jacentes
  - Afficher les règles (onglet d'informations sur les règles)
  - Créer des branches de jeu

- Regardons un tutoriel (en anglais, une liste de 3 vidéos)

36

36

Exemples de résultats d'étudiants - Waterfall

Beaucoup de tentatives de 0 à 50% :

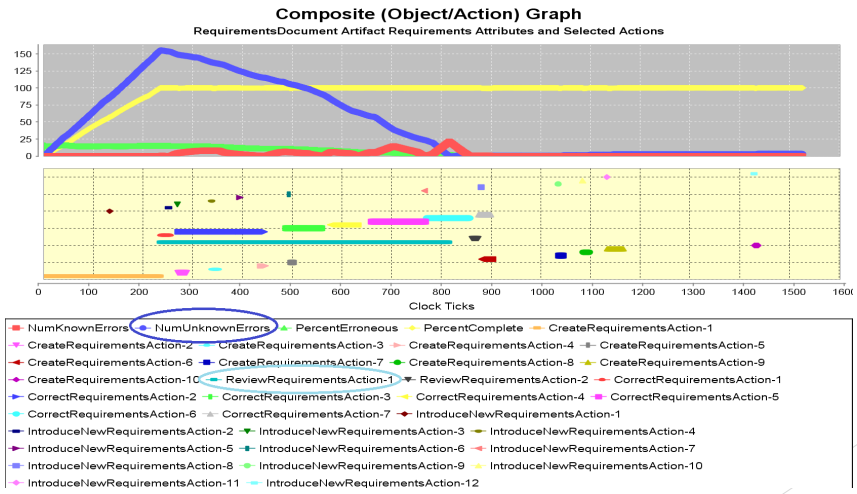
- Trop de tâches assignées simultanément ;
- Pas de bonus, d'augmentation de salaire et de licenciement
- N'a pas attendu un artefact complet avant d'en commencer un autre
- Pas assez de révisions par les bonnes personnes
- N'a pas gardé un œil sur le budget et le temps

37

37

Exemples de résultats d'étudiants - Waterfall

Bénéfices de réviser les artéfacts:



38

38

Exemples de résultats d'étudiants - Waterfall

SEProjects:

Description	Budget	MoneySpent	AllottedTime	TimeUsed	Score
Groceries@Home	294,835.00	283,580.00	1,546	1,480	99

Meilleurs résultats 99/100:

- Nous avons acheté et utilisé tous les outils disponibles ;
- Nous avons affecté uniquement les personnes compétentes aux exigences et tous les autres aux évaluations immédiatement
- Nous avons accordé des bonus à nos meilleurs employés
- Chaque fois que l'utilisateur demandait un changement, nous avons affecté quelqu'un pour effectuer tout le cycle de vie de celui-ci

39

39

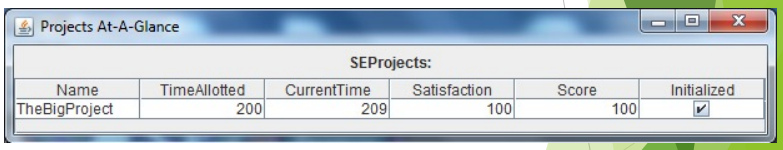
Exemples de résultats d'étudiants - Incrémental

Situation dans SIMSE :

- Le système a été divisé en 4 modules
- Il faut les construire en parallèle et à la fin les intégrer ensemble

Beaucoup de tentatives de 0 à 61% :

- Trop de tâches assignées simultanément;
- Trop d'emphase sur un module;
- Pas assez de feedback demandé au client.



Meilleur résultat 100%:

- Chaque employé travaille sur une tâche à la fois ;
- Petit chevauchement des tâches;
- La précision du travail était très importante;
- Soumettre le travail à l'utilisateur très souvent, ce qui ne nécessite pas de faire une analyse des risques aussi souvent.

40

40

Exemples de résultats d'étudiants - Prototyping

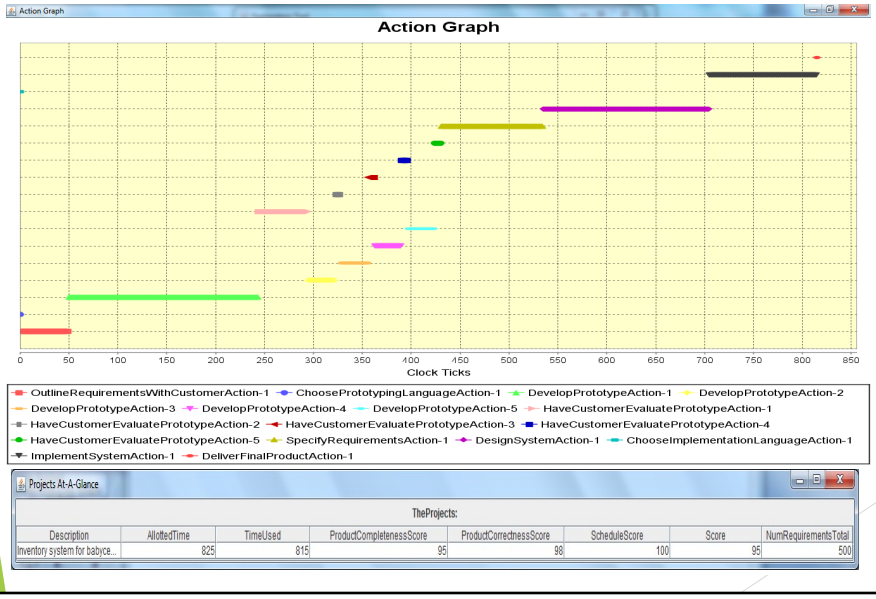
Beaucoup de tentatives de 0% à 70% :

- Gestion du temps;
- Langage utilisé pour le prototypage;
- Combien d'itérations de prototypage doivent être effectuées avant d'implémenter le logiciel réel;
- Comprendre ce que veut le client.

41

41

Exemples de résultats d'étudiants - Prototyping



42

42

Exemples de résultats d'étudiants - RUP

Meilleurs résultats à 90% :

- On a acheté tous les outils;
- Utilisation d'employés ayant beaucoup d'expérience;
- Beaucoup de travail en parallèle;
- Suivre les phases RUP.

43

43

Exemples de résultats d'étudiants - XP

Meilleurs résultats à 63% :

- Les employés doivent travailler en binôme;
- Réaliser des user stories avec les clients;
- Réaliser des exigences, coder, tester à chaque itération;
- Refactoring à chaque itération.

44

44